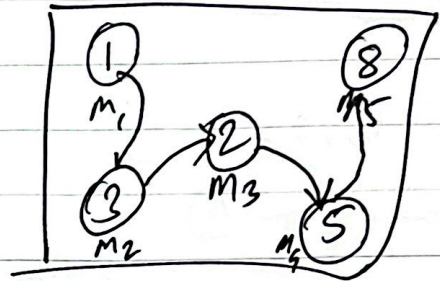
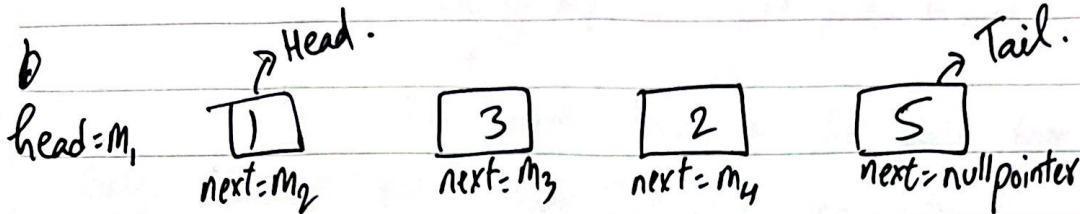


linked list

→ Not in contiguous memory. It can be anywhere on heap memory.

→ Head → starting point of linked list.



Add 8 to linked list.

tail = m_4

then,

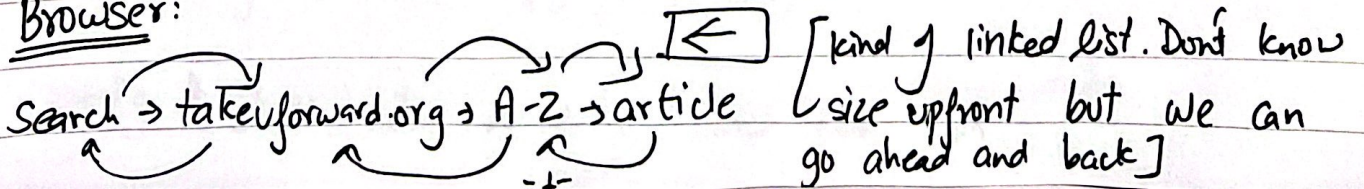
next = null pointer will be updated [next = m_5] then for 8:

↓
next = null pointer

* Where is Linked List used?

Mainly used in stack/queue. In real life we use it in a browser. The LL we talked is ID pointer. We always tracked next ~~is~~ but not previous.

Browser:



linked list: $\square$

Data
pointer to next

Here one data is

$x = 5$
 $\text{int}^* y = \&x$
 $\uparrow$

$z = 2$
 $\text{int}^* \text{ pointer to next}$

This doesn't have a fix data type. So, we need to have
Self defined data type. Thus, we use struct or classes.

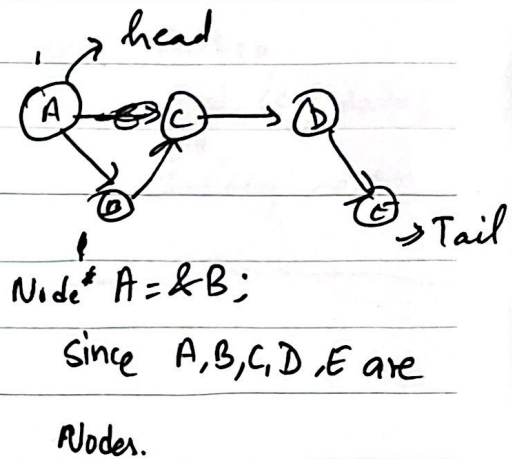
$\rightarrow$ data type [self created]
Struct Node {

int data; $\rightarrow$ has data

Node* next; $\rightarrow$ has pointer to next node.

Node (data, next) {
data = data;
next = next;
} } Constructor

}



Initializing:

Node x = Node(2, null pointer);
Node* y = &x;

$\rightarrow$

2
null ptr

 $\rightarrow$ data
 $\rightarrow$ pointer to next.

OR

Node* y = new Node(2, null pointer)

Node* y stores value of newly created Node with
data $\downarrow$ 2 next $\downarrow$ null ptr.

~~Note~~ Note:

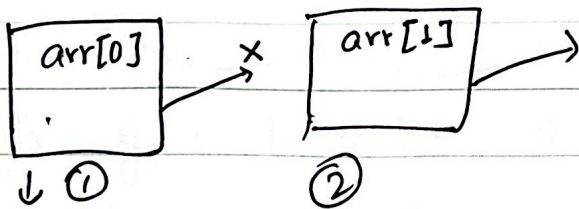
① `Node* y = new Node[arr[0], nullptr];`
and

② `Node y = Node[arr[0], nullptr];`

They are different. One ① is creating a new node and assigning its pointer to y. So `cout << y` gives `0x...f02...`.
But.

② is creating a new object of type Node and assigning it to y. So, `cout << y` will give error. Rather,
`cout << y.data;`
OR,
`cout << y.next;`

How to convert array into linked list:



① first create a node with `data = arr[0]`, `next = nullptr`.

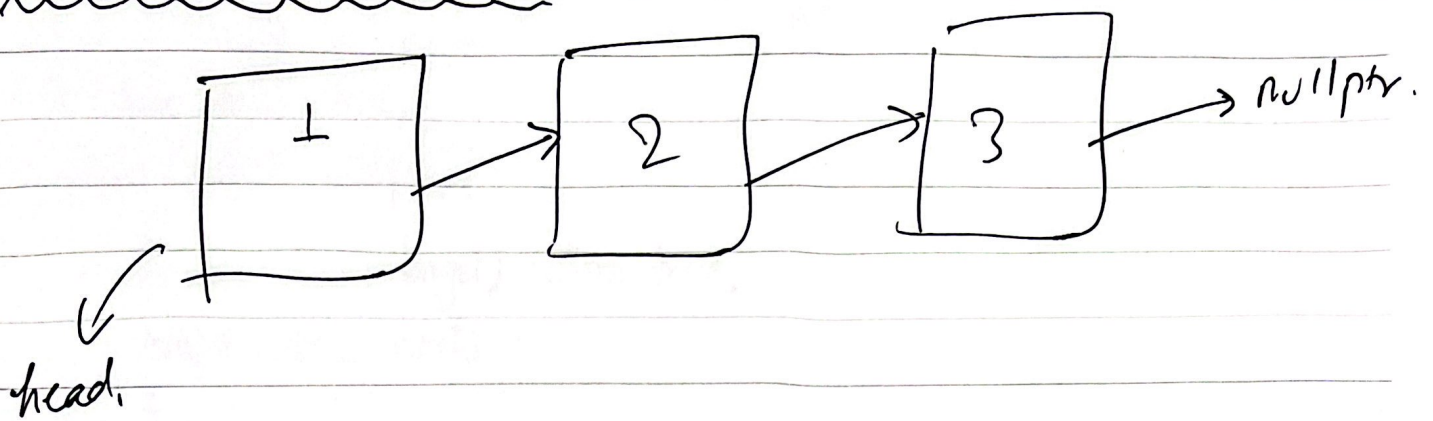
② Then create a new node with `data = arr[1]`, `next = nullptr`.

③ Link ① to ② using pointer. So, you can name ① as `temp` and `mover` as a variable.

`mover = head`
`temp = new node`
then,
`mover = temp.`

```
Node* head = new Node[arr[0]];
Node* mover = head;
for (int i = 1; i < arr.size(); i++) {
    Node* temp = new Node(arr[i]);
    Node* mover = temp;
}
return head;
```

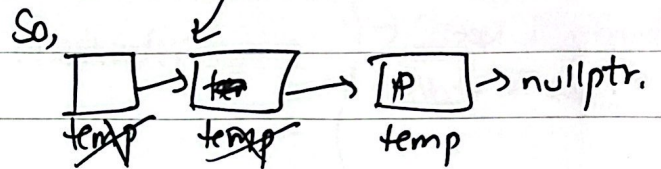
Traversal in linked list:



We never edit the head. Instead create a temporary variable.

```
1) Node* temp = head;
   while (temp != nullptr) {
       cout << temp->data;
       temp = temp->next;
   }
```

→ temp aur data cout karne
→ temp aur temp aur next sangi
jange. [unless its nullptr]



length of a linked list. → $O(N)$ complexity

```
int length of LL (Node* head) {
    int count = 0;
    while (Node* temp = head;
           temp != nullptr) {
        temp = temp->next;
        count++;
    }
    return count;
}
```

Searching in a linked list: $\rightarrow O(n)$

```
bool Search(head, target) {  
    Node* temp = head;  
    while (temp != nullptr) {  
        if (temp->data == target) return true;  
        temp = temp->next;  
    }  
    return false;  
}
```

Summary:

What we did?

- Intro to linked list
- Initializing a node using constructor.
- How to link two nodes
- Traversing in linked list
- Converting an array into a linked list.
- finding length of a linked list.
- Searching in a linked list.

Short Intro

$\rightarrow$ Classes & structs
 $\rightarrow$ Node & Node*